Requirement Management — End-to-End Flow

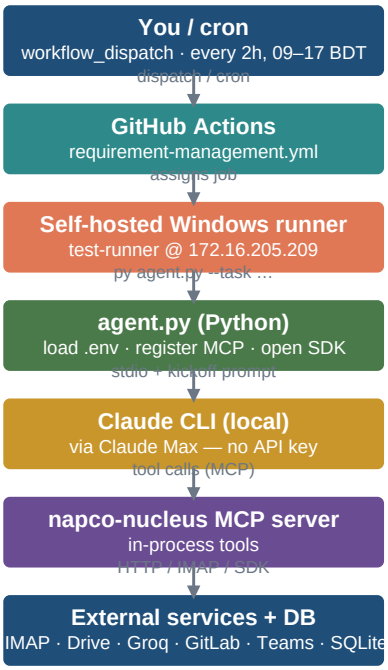
How NAPCO Nucleus turns raw client text into 3-hour GitLab tasks

Mohammad Kamrul Hasan · AI-Augmented QA Architect · April 2026

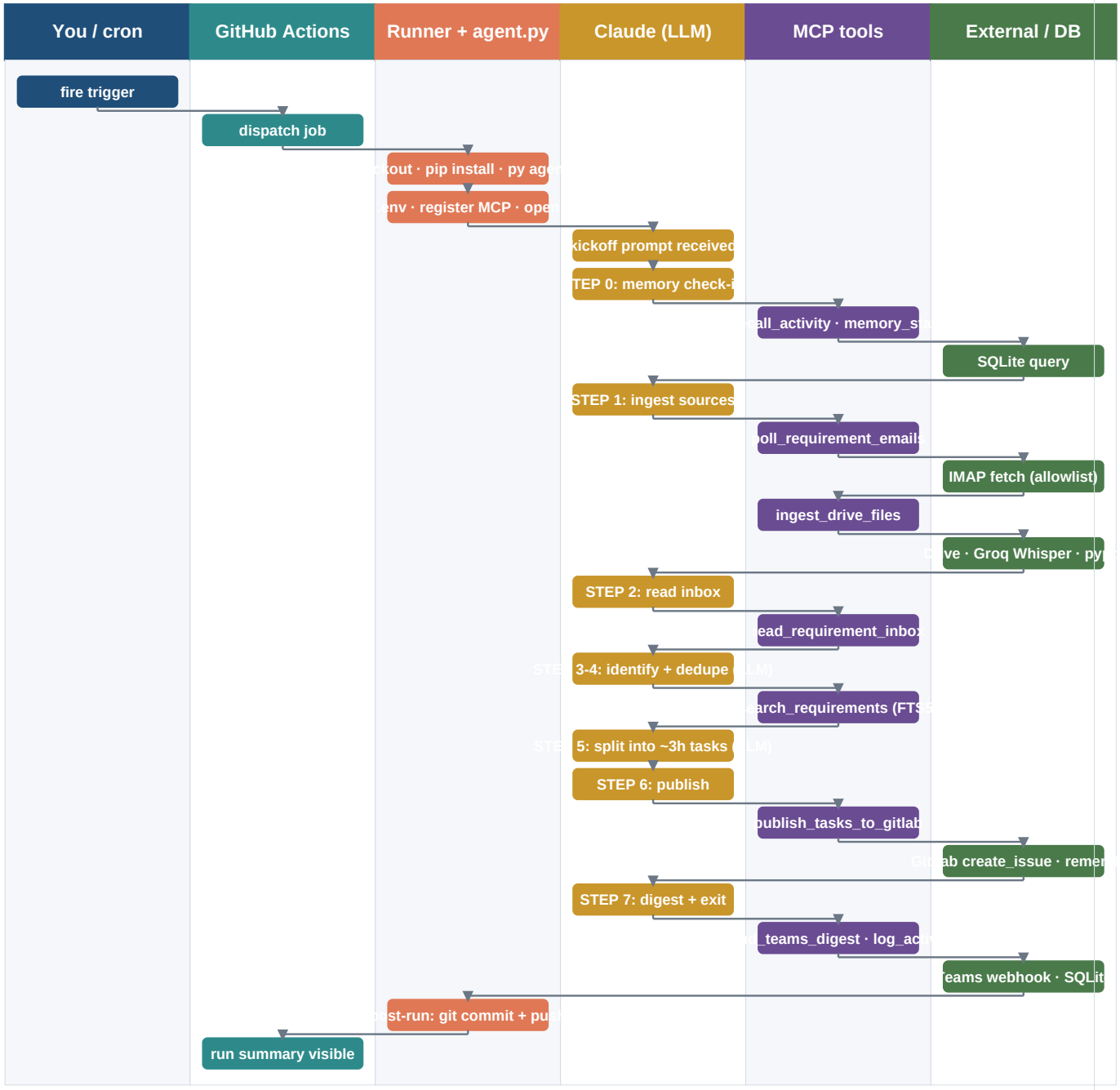
WHAT THIS WORKFLOW DOES

Every two hours during business hours, a Claude-powered agent polls the allowlisted IMAP mailbox, downloads new audio recordings and PDFs from Google Drive, transcribes the audio with Groq Whisper, splits each distinct requirement into ~3-hour tasks, and files them as GitLab issues with two-layer dedup. A short Teams digest is posted after completion if the webhook is configured.

1. ARCHITECTURE — WHO TALKS TO WHOM



2. ONE FULL RUN — SWIMLANE VIEW



3. THE LOOP, STEP BY STEP

0 **Memory check-in (mandatory)**

recall_activity for publish_gitlab and poll_email; memory_stats. Confirms the DB is being written and shows what was published recently.

1 **Ingest new inputs**

poll_requirement_emails fetches IMAP messages from allowlisted senders. ingest_drive_files downloads new Drive audio (→ Groq Whisper) and PDFs (→ pypdf). Idempotent via UIDVALIDITY checkpoint and Drive file-ID tracking.

2 **Read the inbox**

read_requirement_inbox returns every .txt file across email, meetings, chat, documents subfolders. If file_count is 0, stop and report the inbox is empty.

3 **Identify distinct requirements (LLM)**

Claude reads each file and extracts user-visible capabilities, changes, bug fixes, deliverables. Process chatter, greetings, scheduling are ignored.

4 **Dedup against prior work**

search_requirements per candidate against memory.requirements_seen (SQLite + FTS5 fuzzy match). Skip anything with an existing GitLab issue URL.

5 **Split into ~3-hour tasks (LLM)**

Larger requirements split into multiple 3-hour tasks. Smaller related items get merged. Each task gets title, description, acceptance_criteria, estimate_hours, source_ref, optional labels.

6 **Publish to GitLab**

publish_tasks_to_gitlab snapshots the submission, dedupes against open issue titles AND fuzzy memory, creates the rest, writes each created task back to memory with iid + url. Honors NAPCO_NUCLEUS_DRY_RUN.

7 **Digest + exit**

send_teams_digest with one-line summary if TEAMS_WEBHOOK_URL is set. Final log_activity row. Process exits. Runner commits memory + inbox files back to main.

4. WHO COMMANDS WHOM — CHEAT SHEET

Each layer issues one kind of instruction to the layer below it. Claude is the **decider**. MCP tools are the **hands**. GitHub Actions is the **scheduler**. The runner is the **machine**.

Layer	Actor	Command issued	To whom
0	You / cron	workflow_dispatch or schedule fire	GitHub Actions
1	GitHub Actions	run this job	Self-hosted Windows runner
2	Runner shell	py -3 agent.py --task requirement-management	Python agent.py
3	agent.py	client.query("Run the Requirement Mgmt loop now…")	Claude CLI (local, Claude Max)
4	Claude (LLM)	MCP tool calls: poll_emails, ingest_drive, read_inbox, search, publish_digest, log	napco-nucleus MCP server
5	MCP tool fns	HTTP / IMAP / SDK calls	IMAP, Drive, Groq, GitLab, Teams, SQLite
6	Runner shell (post)	git commit && git push	GitHub repo

5. GUARDRAILS

- **Idempotency.** IMAP uses UIDVALIDITY plus since-UID checkpoint. Drive never re-processes a file ID. GitLab dedup runs in two layers: open-title match plus fuzzy match against requirements_seen in memory.
- **Dry-run mode.** workflow_dispatch accepts dry_run=true. Tools check NAPCO_NUCLEUS_DRY_RUN=1 and short-circuit every mutation: no SMTP, no GitLab create, no git push. Memory still logs the dry run.
- **Concurrency.** Workflow group requirement-management with cancel-in-progress=false. Runs queue, never overlap.
- **State persistence.** nucleus_memory.db and data/requirements/ get committed back to main after every run, so the next run inherits the previous run's checkpoints and dedup history.
- **Allowlist.** Only IMAP senders in REQ_SENDER_ALLOWLIST are ingested. Random inbound mail is dropped.
- **Language.** Task titles, descriptions, acceptance criteria are always written in English even when the source is Bangla, Malay, or any other language.